Seralix: A Minimal Language Prototype for Expressing FFT and NTT Algorithms over Exact Integer Arithmetic

David Mateo Barbosa Monsalve
Universidad de los Andes
Systems and Computing Engineering

***Abstract*—This paper presents Seralix, a minimal programming language built from scratch as a proof of capability in language design, parsing, and interpretation. The language was used to implement the Fast Fourier Transform (FFT) and the Number Theoretic Transform (NTT) — two algorithms for efficient polynomial multiplication. A deliberate design goal was to avoid floating-point arithmetic entirely: the FFT runs over exact integer representations of complex numbers, and the NTT uses only modular integer arithmetic. All implementations produce correct results, and measured execution times show clear performance differences between the two approaches. The project demonstrates that a working language pipeline and a family of non-trivial algorithms can both be built from first principles by a single developer.**

I. Introduction

The Fast Fourier Transform is one of the most important algorithms in computing. It reduces the cost of computing a Discrete Fourier Transform from $O(n^2)$ to $O(n \log n)$ [1], and it appears everywhere from signal processing to polynomial multiplication to data compression.

In most settings, FFT is used through a library — NumPy, FFTW, or similar — where the implementation details are hidden. This project takes the opposite approach: instead of using an existing language and library, both the execution environment and the algorithm were built from the ground up.

The execution environment is Seralix, a small interpreted programming language with its own lexer, parser, and runtime. Seralix follows the *interpreted* paradigm: programs are not compiled to machine code or bytecode ahead of time, but instead evaluated directly from the source representation at runtime by a tree-walking interpreter. This design choice prioritises simplicity and transparency of execution over raw performance, making it straightforward to inspect and reason about every step of computation. The algorithms — FFT, NTT, and six variants in total — are written directly in Seralix, using only the constructs the language provides.

A key decision throughout was to avoid floating-point numbers. Standard FFT implementations compute roots of unity as floating-point complex numbers, which introduces rounding error. This project uses two exact alternatives instead:

- **NTT**: replaces complex roots of unity with integer roots computed modulo a prime. All arithmetic stays in whole numbers.
- **Gaussian integer FFT**: represents complex numbers as pairs of integers and performs all arithmetic modulo a carefully chosen prime. The result is exact complex arithmetic with no floating-point involved.

The motivation for the project is simple: it was built to prove it could be done.

II. Problem Statement

The project addresses two problems at once: building a language capable of expressing the algorithms, and implementing the algorithms correctly within it.

A. What FFT and NTT Need

To implement these algorithms, the language must support:

- Functions that call themselves recursively
- Lists that can be read and written by index
- Integer arithmetic, including bitwise operations like shifts and masks
- Loops and conditional branches
- Passing functions as arguments

B. Research Questions

Can a minimal programming language, built from scratch, correctly implement FFT and NTT while preserving $O(n \log n)$ complexity and avoiding floating-point arithmetic?

What is the performance cost of running these algorithms inside a tree-walking interpreter compared to native execution?

III. System Architecture

A Seralix program goes through four stages before producing output:

A. Lexer

The lexer reads raw source code and breaks it into tokens — the smallest meaningful units of the language. Whitespace and comments are discarded. Everything else becomes a named token:

```
# Source
x = 10 + y;

# Token stream
[NAME('x'), OP('='), NUMBER('10'),
 OP('+'), NAME('y'), OP(';')]
```

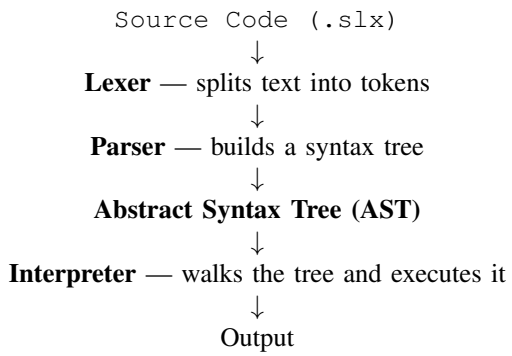

Fig. 1. Execution pipeline of Seralix

Keywords like `if` and `function` are not treated specially by the lexer — they come out as `NAME` tokens and are recognized by the parser. This keeps the lexer simple and general.

B. Parser

The parser takes the token stream and builds an Abstract Syntax Tree. It is written using `funcparserlib` [6], a combinator library, with manual operator precedence handling. Each level of precedence is defined as a rule that folds operators left-to-right, correctly expressing left-associativity:

```
# Multiplication and division (left-
    associative)
multiplicative = (
    unary + many(
        (op('*') | op('/') | op('//') | op('%'
            ))
        + unary
    )
) >> fold_left_into_BinaryOp
```

Calls, member access, and subscript operations are all handled as chains of postfix operations applied left-to-right, so expressions like `a.b[0](x).c` parse and evaluate correctly.

C. Abstract Syntax Tree

The AST is a tree where each node represents one operation or value. Nodes are plain Python data classes. The full set covers:

- **Values**: numbers, strings, booleans, null, lists, dictionaries
- **Operations**: binary operators, unary operators, function calls, member access, subscript access, ternary expressions
- **Statements**: assignment, function definition, return, for-in loop, while loop, if-elif-else, struct/record definition

D. Interpreter

The interpreter walks the AST and evaluates each node. This is the defining characteristic of the *interpreted* paradigm adopted by Seralix: rather than compiling the program to a lower-level representation [7], execution proceeds directly over the tree structure produced by the parser. It maintains a stack of variable environments, one per function call, so that local variables in one function do not interfere with another.

Functions in Seralix are closures: they capture the environment that exists when they are defined, not when they are called. This allows functions defined later in a file to be called from functions defined earlier:

```
closure = self.environ.copy()

def callee(*args, **kwargs):
    # merge: call-time environ provides names
    # declared after this function was defined
        ;
    # closure provides names from definition
        time
    self.stack.append(
        self.environ.copy() | closure.copy()
    )
    ...
    finally:
        self.stack.pop()
```

Compound assignment operators like `+=` and `>>=` are handled by rewriting them at runtime into a plain assignment combined with the corresponding binary operation, keeping the AST simple.

IV. Language Design

Seralix is intentionally small. Every feature in it exists because an algorithm needed it.

A. Functions

```
# Inline lambda
square = (x) -> x * x;

# Named function with a default parameter
function add(x, y=2) {
    return x + y;
}
```

If a parameter with a default value is followed by one without, the parser raises a syntax error before the program runs.

B. Control Flow

```
if x > 0 {
    print("positive");
} elif x == 0 {
    print("zero");
} else {
    print("negative");
}
```

C. Loops

Seralix supports both `while` and `for-in` loops:

```
while i < n {
    i += 1;
}

for item in range(10) {
    result = result + [item];
}
```

It is worth noting that loops are not strictly necessary in an interpreted language that supports tail recursion: any iterative computation can be expressed as a recursive function. Since tail-call optimisation is a post-AST runtime decision, its presence or absence is not a property of the language grammar itself. Loops are included in Seralix for clarity and practical convenience, not as a fundamental requirement.

D. Data Structures

```
# List with indexed assignment
xs = [1, 2, 3];
xs[0] = 42;

# Slice access (used in FFT)
evens = xs[0::2];
odds  = xs[1::2];

# User-defined types
struct Point(x, y);       # mutable
record Color(r, g, b);    # immutable
```

E. Operators

The full operator set is supported: arithmetic, bitwise (`&`, `|`, `^`, `<<`, `>>`), comparison, boolean, ternary (`? :`), and all compound assignment forms. Precedence is enforced by the grammar, not at runtime.

V. Theory: Why FFT Works and How NTT Improves It

A. The DFT and Its Cost

The Discrete Fourier Transform of a sequence $a_0, \ldots, a_{n-1}$ is:

$$A_k = \sum_{j=0}^{n-1} a_j \, \omega_n^{jk} \tag{1}$$

where $\omega_n = e^{2\pi i/n}$ is a complex root of unity. Computing this directly for every k costs $O(n^2)$ multiplications [2].

B. FFT: Divide and Conquer

The Cooley-Tukey FFT splits the sequence into even and odd indexed elements and computes two smaller transforms:

$$A_k = A_k^{(\text{even})} + \omega_n^k \cdot A_k^{(\text{odd})} \tag{2}$$

$$A_{k+n/2} = A_k^{(\text{even})} - \omega_n^k \cdot A_k^{(\text{odd})} \tag{3}$$

This halving recurrence gives $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$.

C. NTT: Exact Integer Version

The NTT replaces the floating-point complex root ω_n with an integer root computed modulo a prime p. If p has a primitive root g and n divides $p-1$, then:

$$\omega_n \equiv g^{(p-1)/n} \pmod{p} \tag{4}$$

satisfies $\omega_n^n \equiv 1 \pmod{p}$, which is all the butterfly needs. This project uses $p = 998244353$ with primitive root $g = 3$, a standard choice in competitive programming [5] that supports transform sizes up to 2^{23}.

D. Gaussian Integer FFT: Exact Complex Version

For cases where actual complex structure matters, this project also implements an FFT over Gaussian integers modulo a prime [4]. A Gaussian integer is a complex number $a + bi$ where both a and b are integers. When working modulo a prime p where $p \equiv 1 \pmod 4$, a square root of -1 exists modulo p, which means we can represent i exactly as an integer. Complex numbers then become pairs of integers $[re, im]$, and all arithmetic stays exact:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i \tag{5}$$

The butterfly structure of the FFT is identical to the NTT — the only difference is that each multiplication uses Gaussian integer arithmetic instead of scalar modular arithmetic. The primitive complex root of unity $\omega_n = e^{2\pi i/n}$ is represented exactly as a pair $[\cos_n, \sin_n]$ computed modulo p.

This approach carries specific constraints on prime selection. The prime p must satisfy both $p \equiv 1 \pmod 4$ (to guarantee existence of $\sqrt{-1}$) and $n \mid (p-1)$ (to guarantee existence of n-th roots of unity). In practice, standard NTT-friendly primes such as $p = 998244353 = 119 \cdot 2^{23} + 1$ already satisfy both conditions for power-of-two transform sizes. For arbitrary n, a prime of the form $k \cdot \text{lcm}(4, n) + 1$ must be located. Current standard choices in the literature use primes in the range of 30–64 bits, balancing the need for large enough moduli to avoid coefficient overflow against the cost of 64-bit modular arithmetic. For the polynomial sizes tested in this project, a 30-bit prime is sufficient; larger inputs would require either a larger prime or a multi-modulus approach with Chinese Remainder Theorem reconstruction.

VI. Implementation

A. Recursive FFT

The language was first validated using a recursive FFT, which directly expresses the divide-and-conquer structure:

```
function fft(a, invert=false) {
    if len(a) == 1 {
        return [a[0]];
    }
    even = fft(a[0::2], invert);
    odd  = fft(a[1::2], invert);
    # butterfly recombination over even and
        odd...
}
```

This confirms that the language handles recursion, slice indexing, and list manipulation correctly.

B. Iterative NTT

The production NTT uses an iterative butterfly with bit-reversal permutation, which avoids the overhead of recursion. The key insight is that the primitive root of unity for any valid transform size can be derived in a single modular exponentiation — no search needed:

```
# g^{(p-1)/n} mod p gives the primitive n-th
    root
function ntt_root(n, m, g) {
    return modpow(g, (m - 1) // n, m);
}

function ntt_core(xs, invert, w, m) {
    n      = len(xs);
    result = bitrev(xs);
    if invert == 1 { w = modinv_prime(w, m); }
    length = 2;
    for i in range(32) {
        if length <= n {
            w_len = modpow(w, n // length, m);
            j = 0;
            for k in range(n // length) {
                ww = 1;
                for l in range(length // 2) {
                    u = result[j + l];
                    v = result[j + l + length
                        // 2]
                            * ww % m;
                    result[j + l] = (u + v) %
                        m;
                    result[j + l + length //
                        2]
                            = (u - v + m) % m;
                    ww = ww * w_len % m;
                }
                j = j + length;
            }
            length = length * 2;
        }
    }
    if invert == 1 {
        inv_n = modinv_prime(n, m);
        for i in range(n) {
            result[i] = result[i] * inv_n % m;
        }
    }
    return result;
}
```

C. Gaussian Integer FFT

The Gaussian integer FFT has the same butterfly structure as the NTT, but each multiplication is replaced with the complex version:

```
function gauss_mul(x, y, p) {
    re = (x[0]*y[0] - x[1]*y[1]) % p;
    im = (x[0]*y[1] + x[1]*y[0]) % p;
    return [re, im];
}
```

After the inverse transform, output values are mapped back from $\{0, \ldots, p-1\}$ to actual integers using centered reduction: any value greater than $p/2$ is replaced by value $- p$.

D. Polynomial Multiplication

All transform variants are used for polynomial multiplication via the same three-step process:

```
function polymul_ntt(xs, ys) {
    sz = next_pow2(len(xs) + len(ys) - 1);
    w  = ntt_root(sz, MOD, PRIM_ROOT);
    # 1. Forward transform both inputs
    xa = ntt_core(pad(xs, sz), 0, w, MOD);
    ya = ntt_core(pad(ys, sz), 0, w, MOD);
    # 2. Pointwise multiply in transform
        domain
    za = [];
    for i in range(sz) {
        za = za + [xa[i] * ya[i] % MOD];
    }
    # 3. Inverse transform to get the product
    return trim(ntt_core(za, 1, w, MOD));
}
```

Both transforms also support arbitrary root orders: if the default prime does not support the requested transform size, the implementation finds a suitable prime automatically and derives the correct root in one step.

VII. Results and Validation

All implementations were tested against the known product:

$$(1+2x+3x^2)(4+5x+6x^2) = 4+13x+28x^2+27x^3+18x^4 \tag{6}$$

```
a = [1, 2, 3];
b = [4, 5, 6];

pprint(polymul_ntt(a, b));
# Output: [4, 13, 28, 27, 18]

pprint(polymul_fft(a, b));
# Output: [4, 13, 28, 27, 18]
```

All six variants — NTT standard, NTT with $n = 8$, NTT with $n = 16$, Gaussian FFT standard, Gaussian FFT with $n = 8$, Gaussian FFT with $n = 16$ — produce identical output, cross-validated against the naïve $O(n^2)$ baseline.

TABLE I
Measured execution times for polynomial multiplication of `[1,2,3]` × `[4,5,6]`

Method	Time (ms)
`polymul_fft` (Gaussian FFT)	40.1
`polymul_fft_arbitrary` ($n = 8$)	42.4
`polymul_fft_arbitrary` ($n = 16$)	41.2
`polymul_ntt` (standard)	18.9
`polymul_ntt_arbitrary` ($n = 8$)	18.7
`polymul_ntt_arbitrary` ($n = 16$)	18.7

VIII. Discussion

A. What the Results Show

The language correctly handles everything the algorithms need. Specifically:

- **Recursion**: the recursive FFT calls itself on even and odd subsequences without issue across all tested input sizes.
- **List indexing and assignment**: bit-reversal permutation and butterfly updates require in-place list writes by index, all of which execute correctly.
- **Bitwise arithmetic**: the bit-reversal step uses left shift, right shift, and bitwise OR; all produce correct results.
- **Nested loops**: the three-level butterfly loop (over stages, groups, and positions) executes without error.
- **Closures**: the NTT and FFT functions reference module-level constants (`MOD`, `PRIM_ROOT`) captured at definition time, which are correctly resolved at call time.
- **Higher-order behaviour**: helper functions such as `modpow` and `modinv_prime` are passed around and called correctly as first-class values.

The syntax proved sufficient for expressing a complete family of transform algorithms without any workarounds or runtime patches.

B. Why NTT Is Faster Than Gaussian FFT

NTT is about twice as fast as the Gaussian FFT. The reason is straightforward: each NTT butterfly step requires one modular multiplication, while each Gaussian FFT butterfly requires the equivalent of four — two multiplications and two additions to compute the real and imaginary parts separately. The extra work per step adds up across all butterfly stages.

The arbitrary root variants show no measurable overhead because the default prime $p = 998244353$ already supports transform sizes of 8 and 16. The root is computed in a single operation and no prime search is needed.

C. Interpreter Overhead

The millisecond-scale execution times are much slower than what native Python would take for the same computation — which would be in the microseconds. This gap comes from the cost of tree-walking interpretation: every operation requires a Python function call, a dictionary lookup in the environment stack, and pattern matching on the AST node type. This is expected behavior for a tree-walking interpreter and does not reflect a flaw in the algorithm. The $O(n \log n)$ structure is intact; the constant factor is the price of interpretation.

D. Floating-Point Limitation Was Overcome

Both the NTT and the Gaussian integer FFT produce exact integer results with no rounding at any stage. The standard FFT approach — computing $e^{2\pi ik/n}$ as a floating-point number — was deliberately avoided from the start. An earlier version of this work listed floating-point precision as an open limitation; it is not a limitation of this implementation.

E. Remaining Limitations

- **List append performance**: adding an element to a list via `result + [x]` copies the entire list each time, making repeated appends $O(n^2)$ in the interpreter. For the small inputs tested this is not observable, but it becomes the dominant cost for large polynomial degrees.
- **No caching**: functions like the cyclotomic polynomial builder recompute their result on every call, even when the input has not changed. Memoisation would require a native dictionary type at module scope.
- **No type checking**: passing a non-prime as a modulus, for example, produces arithmetically incorrect output rather than an early error. There is no mechanism to detect this class of mistake before execution.
- **Stack depth**: deeply recursive calls are bounded by Python's own recursion limit (default 1000 frames). For the recursive FFT this becomes critical at input sizes of $n = 2^{10} = 1024$ and above, where the recursion depth equals $\log_2 n = 10$ levels but each level spawns two sub-calls, exhausting the stack rapidly. The iterative NTT avoids this entirely; the recursive FFT is therefore suitable only for small inputs or after raising Python's recursion limit explicitly.

IX. Conclusion

This project demonstrates that a minimal programming language can be designed and built from first principles, and that non-trivial algorithms — FFT, NTT, and their exact arithmetic variants — can be correctly implemented and executed within it.

Seralix provides a working interpreter pipeline, supports the full set of constructs needed to express these algorithms, and produces verified correct results. The performance measurements are consistent with what a tree-walking interpreter should produce, and the algorithmic structure is preserved at $O(n \log n)$.

More than the specific algorithms, the project validates the process: a language was designed, built, and used to solve a real computational problem, entirely from scratch.

X. Future Work

- **Static type system**: catch parameter and type errors at parse time rather than during execution.
- **Module system**: allow splitting programs across multiple `.slx` files.
- **In-place list mutation**: replace list concatenation with true append to make large-input algorithms practical.
- **Compiled backend**: generate Python bytecode or C from the AST to remove interpreter overhead while keeping the language model intact.
- **Cyclotomic integer transforms**: the algebraic infrastructure for exact arithmetic in $\mathbb{Z}[\omega_n]$ is already implemented in Seralix; connecting it to the butterfly kernel is a natural next step.

REFERENCES

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022, ch. 30.

[3] D. Harvey and J. van der Hoeven, "Integer multiplication in time $O(n \log n)$," *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.

[4] R. Crandall and B. Fagin, "Discrete weighted transforms and large-integer arithmetic," *Mathematics of Computation*, vol. 62, no. 205, pp. 305–324, 1994.

[5] A. Laaksonen, *Competitive Programmer's Handbook*. Self-published, 2018, ch. 30 (Number Theory), available online.

[6] V. Vlasov, `funcparserlib`: Recursive descent parser library for Python, version 1.0.1, 2022. [Online]. Available: https://github.com/vlasovskikh/funcparserlib

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Pearson/Addison-Wesley, 2006.